CHAPTER 08 DATA ENGINEERING / INCREMENTAL SYSTEMS

Streams, Tasks, and Dynamic Tables

Make changed data useful without rebuilding the world

A correction is not another megawatt-hour.
An operator opens the Finland Energy Analytics dashboard at 10:15. Wind generation for 10:00 is shown as 12 MWh, although a corrected reading of 14 MWh arrived five minutes earlier. A naive pipeline appends both observations to a daily aggregate and displays 26 MWh. The ingestion service did its job, but the analytical result is wrong. The system needs not only rapid delivery, but an unambiguous rule for *which version of a business fact is current*, together with a dependable method for maintaining that rule as changes arrive.

Chapter 7 established a safe bronze ingestion boundary: events carry identity, UTC interval, revisions, and records of retry and validation. This chapter consumes that boundary. We compare two styles of downstream maintenance: *procedural* change consumption using Snowflake Streams and Tasks, and *declarative* materialisation using Dynamic Tables. Both solve meaningful problems; neither can repair an unspecified business contract.

Learning outcomes
After this chapter, you should be able to: (1) reason about insert, update, delete and replay semantics; (2) explain stream offsets, transactional consumption and staleness; (3) build a deterministic MERGE with a triggered Task; (4) specify a Dynamic Table with an intentional refresh mode and target lag; (5) compare consistency, freshness, observability, cost and recovery; and (6) reconcile two implementations on a controlled energy fixture.

Prerequisites. Chapters 5–7, SQL joins and window functions from Chapter 4, and a basic understanding of change data capture. The offline laboratory uses Python 3.10+ without external packages; SQL templates require an authorised Snowflake account and have *not* been executed there.

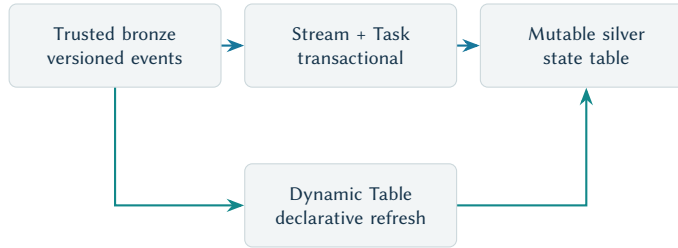


Figure 8.1. Alternative paths to a comparable current-state view; not one pipeline requiring both.

8.1 Change data capture and incremental updates

8.1.1 Change data capture as an information contract

Change data capture (CDC) describes changes to a source dataset rather than transmitting a complete current snapshot after every alteration. In a database log, CDC may identify committed INSERT, UPDATE and DELETE operations. In an event feed, what looks like CDC can instead be a collection of immutable *revision events*: the original measurement remains in the log and a later event states a better value. These two encodings are not automatically interchangeable. Snowflake Streams expose change tracking over a supported source object; they do *not* magically supply business meaning when an upstream event is a correction rather than a SQL UPDATE.

For the energy example, define business key $K = (provider, technology, interval_utc)$. The provider identifies the source authority, technology identifies the energy category and the interval represents a UTC measurement grain, not a wall-clock label in Europe/Helsinki. A new event contains `revision`, `event_id`, `operation` and its measurement. The business contract is *highest approved revision wins*. This is different from last record received wins: delayed networks may deliver a newer revision first and an obsolete correction later.

Time	Message	Required result
10:00	Wind r1: 12	Current wind = 12 MWh
10:05	Wind r2: 14	Replace; current wind = 14 MWh
10:06	Wind r1: 12	Ignore obsolete replay; still 14 MWh
10:07	Wind r2: 14	Duplicate: one effective version

Table 8.1. Arrival order and revision order are not equivalent.

A CDC feed should publish a documented rule for transaction boundaries, primary keys, deletes and retention. Without it, two consumers can independently produce incompatible totals while both pipelines appear operational. The rule for energy is especially important: summing raw received versions creates double-counting, so the silver table must resolve state before an analytical aggregate is calculated.

Decision test
Is the source a database table emitting row-level mutations, an append-only audit log, or a revisioned measurement API? Identify the difference first. If you cannot state what a DELETE means for a business key and whether the next lower revision may reappear, do not choose the ingestion operator yet.

Project reference: P081: PostgreSQL CDC to Snowflake reference. *Not independently executed; rights review pending.*

Check. Which key prevents the 10:00 wind reading from being counted twice, and why is `received_at` an inadequate sole conflict-resolution field?

8.1.2 Inserts, updates, deletes, and tombstones

In a state table, INSERT creates a previously unknown row, UPDATE changes attributes of an existing row, and DELETE removes it from that table's current state. Yet these physical operations do not necessarily preserve an audit trail. The bronze event log should normally retain the original delivery for traceability, within an approved retention and privacy policy. Silver can store one current state per key, with a flag or operation marker indicating that the latest approved revision is a deletion.

A *tombstone* is an explicit deletion record whose version remains visible to the maintenance logic. Suppose TEST energy has revision 1 with value 1 MWh and revision 2 marked DELETE. Removing the silver row entirely looks correct until an old revision 1 is replayed into an empty target; the row may be reinserted. A retained revision 2 tombstone prevents such resurrection. Downstream visible views exclude tombstones; reconciliation retains them.

Listing 1: State representation, not a complete executable pipeline.

```
CREATE TABLE CH08_SILVER_STATE (
  provider VARCHAR, technology VARCHAR,
  interval_utc TIMESTAMP_NTZ,
  revision NUMBER, energy_mwh NUMBER(12,3),
  operation VARCHAR, event_id VARCHAR,
  ingested_at TIMESTAMP_NTZ
);
-- The consumer-facing view must exclude DELETE state.
SELECT technology, energy_mwh
FROM CH08_SILVER_STATE
WHERE operation = 'UPSERT';
```

Soft delete versus hard delete. A soft delete preserves the tombstone in the current-state table and hides it from ordinary consumers. A hard delete may be required by a separate retention or erasure policy, but then a version ledger or upstream snapshot reconstitution must prevent reintroduction. Do not confuse technical event replay with the legal requirements for personal-data erasure; these are different governance decisions.

Corrections are not negative deltas by default. If a reading changes from 12 to 14 MWh, the corrected *state* is 14 MWh. A delta-based feed might instead publish +2 MWh, but that is a different contract. This chapter uses absolute-value revisions; downstream aggregates sum the latest approved states rather than raw change amounts.

> **Invariant worth defending**
> For every business key there is at most one latest approved state; deleting a key never allows an older revision to become current. Implementations may differ internally, but their published view must satisfy this invariant after recovery and replay.

Check. If the only record retained after a DELETE is the absence of a row, what additional durable information is needed to reject an old INSERT?

8.1.3 Incremental correctness and replay assumptions

Incremental processing is attractive because it touches only what changed, but correctness depends on more than an efficient execution plan. Assume a source may deliver a message at least once; ingestion may retry after an ambiguous acknowledgment, while later corrections may arrive out of order. A replay-safe consumer requires deterministic precedence, stable keys and an explicit commit boundary. If only the target update commits while a consumer cursor advances too early, data can be lost. If only the target commits and the cursor does not advance, replay must be safe.

Separate three identities: a *delivery identity* distinguishes transport attempts, an *event identity* names a logical source version, and a *business key* identifies the fact to present. The first may change on retry; the second should remain stable; the third intentionally groups multiple revisions. Reliable deduplication depends on choosing the correct identity for each layer. A SHA-256 file checksum from Chapter 5 is useful for input integrity, but it is not the business identity of an energy reading.

Acceptance criteria for the running case
(1) A duplicate event may increase bronze physical rows but not silver business totals. (2) A higher revision updates exactly one key. (3) A lower revision cannot roll the result back. (4) A tombstone hides the key and blocks resurrection. (5) A rejected negative value is quarantined instead of changing an approved row. (6) Reprocessing an already committed batch leaves the approved result unchanged.

An *incremental equivalence test* compares the incremental result with a reference snapshot recomputed from all *accepted* events. Let E be the accepted event history, $F(E)$ the current-state result obtained by full reduction, and $I(E_1, \ldots, E_n)$ the state after sequential incremental batches. The requirement is

$$I(E_1, \ldots, E_n) = F\left(\bigcup_{j=1}^{n} E_j\right)$$

for the stated conflict-resolution contract. This equality is about logical records, not matching warehouse execution plans, scheduling decisions or costs. The offline companion tests it over synthetic events; it does not simulate Snowflake's internal stream metadata.

An additional assumption must be stated: if a provider can issue two conflicting events with the same revision, precedence requires a reproducible tie-breaker, preferably specified by the source. This book uses the persisted UTC ingest timestamp and event ID. In production, investigate why conflicting same-revision values were emitted instead of silently accepting arbitrary arrival order. **Decision check.** Would a `MERGE` keyed on `event_id` maintain the correct current energy state? Explain why the *business key* belongs in the target match condition.

Project reference: P082: Incremental data warehouse community study. *Not independently executed; rights review pending.*

8.2 Change tracking with Streams

8.2.1 Stream offsets, source objects, and consumption

A Snowflake Stream is a change-tracking object over a supported source table or view. It tracks a *position* in the source's version history and returns the *net changes between offsets* as a queryable relation. It is not a separate queue storing one independent copy of every historical event. A stream on an append-only event log can be configured `APPEND_ONLY = TRUE` to expose inserted records; a standard stream is appropriate when source row updates and deletes must be captured.

Listing 2: Create an insert-only change boundary for an append-only source.

```
CREATE STREAM CH08_BRONZE_STREAM
  ON TABLE CH08_BRONZE_EVENTS
  APPEND_ONLY = TRUE;
SELECT event_id, revision, METADATA$ACTION
FROM CH08_BRONZE_STREAM; -- observation only
```

Crucially, a `SELECT` from a stream does *not* consume its offset. Successful DML using the stream in a transaction advances that stream's offset when the transaction commits; a rolled-back transaction does not. For an append-only source, reading a stream in a target `MERGE` is a common transactional boundary: the target data and the stream offset move together after success. If multiple independent consumers are needed, give each its own stream and monitor it separately.

8.2.2 Metadata columns and change semantics

Standard streams expose action metadata (INSERT or DELETE), an update marker for change pairs, and a source-row identifier. Their exact column names are documented in the SQL reference. An UPDATE on the source table is commonly represented as linked DELETE and INSERT entries; that does *not* mean that an immutable measurement revision appears as an UPDATE. A stream on our append-only bronze log returns INSERTs for new *events*, including business corrections and tombstones. Interpretation of `operation='DELETE'` remains our *payload contract*.

Common misconception
Two consecutive stream queries can return identical rows because observation alone does not commit consumption. Snowflake row tracking IDs are also not stable business keys. The first belongs to Snowflake source-row tracking; the second must come from the domain model.

Check. Why can an incoming event with `operation='DELETE'` still have stream metadata `METADATA$ACTION='INSERT'`?

8.2.3 Retention, stale streams, and safe recovery

A stream's offset depends on historical change tracking being retained on its source. If the consumer falls behind beyond the effective data retention window, a stream may become *stale*; it can no longer safely reconstruct changes since its saved offset. Monitor SHOW STREAMS or DESCRIBE STREAM, including STALE_AFTER, and set alarms well before the predicted threshold. Do not treat the displayed timestamp as a perpetual guarantee independent of retention configuration, cloning or source replacement.

Listing 3: Inspect stream status and potential staleness.

```
SHOW STREAMS LIKE 'CH08_BRONZE_STREAM';
SELECT SYSTEM$STREAM_HAS_DATA(
  'CH08_BRONZE_STREAM') AS possible_change;
```

The system predicate is intended for Task gating. It is designed to avoid *false negatives*, but may report TRUE for an effectively empty net change set. A task that continually reacts to such a result without consuming the stream can waste credits. A committed DML operation consuming the empty delta advances its offset; investigate the situation rather than repeatedly resuming tasks without a plan.

Operational recovery playbook: pause the affected task, preserve diagnostic metadata, identify the last committed business watermark and source version, then determine whether the source event log or an earlier trustworthy snapshot can reconstruct the required interval. Establish a new stream boundary only after reconciling the rebuilt target with the source of truth. Simply dropping and recreating the stream loses the original offset; it is not evidence that missing events were recovered.

Staleness budget
Retention and outage tolerance must be designed together. A source retaining sufficient history for only a short period is incompatible with a longer unattended recovery target. Calculate monitoring lead time, manual response window and worst-case backfill duration; test these before calling the system recoverable.

For a prolonged outage, a complete accepted-event archive in durable external storage is a valuable independent recovery source. The chapter's small offline fixture acts as that archive only for illustration; it is not a substitute for an actual audited retention plan. **Check.** Does SYSTEM$STREAM_HAS_DATA returning TRUE prove that at least one publishable energy row exists? Why not?

Project reference: P083: Snowflake-Labs stream-creation utility. *Not independently executed; rights review pending.*

8.3 Scheduling jobs and task dependency graphs

8.3.1 Scheduled and triggered tasks

A Task runs SQL statements or supported procedures on user-managed or serverless compute. A root task can run on a schedule; a triggered task can wake when a stream reports possible changes. A schedule represents an execution opportunity, *not* a guarantee of eventual business freshness: a long-running query, paused warehouse, restricted role or failed prerequisite may cause its result to arrive late. Choose a trigger for irregular incoming events, and a fixed schedule when a batch contract or predictable reporting window matters.

Listing 4: An initially suspended triggered Task; adapt warehouse before use.

```
CREATE TASK CH08_ENERGY_MERGE_TASK
  WAREHOUSE = CH08_WH
  WHEN SYSTEM$STREAM_HAS_DATA('CH08_BRONZE_STREAM')
AS
  MERGE INTO CH08_SILVER_STATE ... ;
-- The full executable MERGE is in the lab SQL file.
-- ALTER TASK CH08_ENERGY_MERGE_TASK RESUME;
```

The ellipsis is a *teaching placeholder*, not executable SQL; the companion file supplies the complete Task statement. A newly created task is suspended by default. Resuming it can initiate billable processing. For a serverless triggered task, the current creation syntax requires `TARGET_COMPLETION_INTERVAL` instead of a `WAREHOUSE`; its billing and scaling characteristics differ from a user-managed warehouse.

8.3.2 Dependencies and failure domains

A task graph expresses causal order with `AFTER` predecessor relationships. A sensible chain might *load approved bronze* $\rightarrow$ *merge silver* $\rightarrow$ *validate published metrics*. Every arrow must encode a genuine dependency. A downstream task should not publish data if a required predecessor failed. Multiple parents, finalizer tasks and overlapping graph runs create additional semantics that need testing on the relevant account edition and service version.

8.3.3 Monitoring, retries, and runaway schedules

Capture task execution ID, start/end timestamps, state, error messages and number of affected business keys. Set a policy for repeated failures, suspension and operator notification. Re-execution is not safe merely because a Task can be retried: any external call or non-idempotent side effect in the body still needs its own retry contract.

Design rule
If the silver merge finishes in 20 seconds but a reconciliation query takes five minutes, they should not necessarily share one continuously triggered transaction. Separate the correctness-critical commit from slower diagnostic work, and specify how the validation observes the committed state.

Project reference: P084: Task graphs getting-started guide. *Not independently executed; rights review pending.*

8.4 Incremental MERGE and upsert design

8.4.1 Match conditions determine the meaning of a row

The MERGE command compares each source row to the target using an explicit match condition. For our energy state, `provider`, `technology` and `interval_utc` determine the current row. `revision` must *not* be included in the match key: doing so would insert a new state row for every correction. `event_id` is likewise an audit identity, not a key for the current business fact.

8.4.2 Deterministic deduplication before matching

Snowflake documents that a MERGE can be nondeterministic when several source rows match the same target and request conflicting actions. The default protection against nondeterministic MERGE results detects some conflicts, but duplicate source rows can also produce duplicate INSERTs into an empty target. Reduce the incoming batch to one chosen event per business key *before* the match. A ranking function can encode the same revision and tie-break policy used in the full-state reference.

Listing 5: Core of the laboratory's deterministic source reduction.

```
SELECT event_id, provider, technology,
       interval_utc, revision, energy_mwh,
       operation, ingested_at
FROM CH08_BRONZE_STREAM
WHERE METADATA$ACTION = 'INSERT'
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY provider, technology, interval_utc
  ORDER BY revision DESC, ingested_at DESC,
           event_id DESC
) = 1;
```

8.4.3 Idempotency, version tests, and tombstones

The target update should proceed only when the incoming version is strictly better according to the stated ordering. The DELETE payload is stored as an updated tombstone, not a physical deletion that would allow an older revision to return. Reject malformed or negative measurements in an upstream quality gate; otherwise a filtered source may be *consumed* without ever reaching a quarantine table. In a real pipeline, a single transaction can stage invalid and valid rows before committing the final state; the textbook SQL assumes bronze contains already-approved events.

Testing requirement
Insert a row, apply a correction, replay both deliveries, then present a deletion and an older replay. Assert one stable latest state per key and a stable aggregate. Matching row counts alone cannot establish correctness.

Project reference: P085: Streams-and-Tasks ETL example. *Not independently executed; rights review pending.*

Project reference: P086: SCD Type 1 upsert example. *Not independently executed; rights review pending.*

8.5 Defining transformations with Dynamic Tables

8.5.1 A query definition instead of a manual update loop

A Dynamic Table stores the materialised result of a SQL definition and is maintained by Snowflake according to supported refresh policies. The author declares *what the result means*: for every energy key, retain the most recent approved revision. Snowflake manages the update plan and dependencies. This can remove procedural bookkeeping for transformations that are naturally expressible as SQL, but it does not make source contracts, semantic validation or operational responsibilities disappear.

Listing 6: Declaration of a latest-revision materialisation; requires approved warehouse.

```
CREATE OR REPLACE DYNAMIC TABLE CH08_DT_LATEST
  TARGET_LAG = '20 minutes'
  WAREHOUSE = CH08_WH
  REFRESH_MODE = INCREMENTAL
AS
SELECT event_id, provider, technology,
       interval_utc, revision, energy_mwh,
       operation, ingested_at
FROM CH08_BRONZE_EVENTS
WHERE revision >= 0
  AND (operation = 'DELETE' OR
       (operation = 'UPSERT' AND energy_mwh >= 0))
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY provider, technology, interval_utc
  ORDER BY revision DESC, ingested_at DESC,
           event_id DESC
) = 1;
```

8.5.2 Declarative semantics and consumer views

The declaration yields *latest state including tombstones*. A simple ordinary View over the dynamic result exposes only `operation='UPSERT'` to reports. Importantly, a Dynamic Table is not a writable ordinary table; a consumer cannot arbitrarily `UPDATE` its rows to repair one reading. Repair the source history or change the definition and allow refresh to propagate. Direct DML and stored-procedure side effects require a different procedural boundary.

For this source, `QUALIFY ROW_NUMBER() ... = 1` is documented as a supported and often efficient incremental-refresh pattern. Nevertheless, creating a dynamic table with an explicit unsupported construct returns an error rather than silently changing the requested refresh mode. Always capture the actual refresh mode from `SHOW DYNAMIC TABLES`, and check service feature availability for the account.

Economic consequence
Creating a dynamic table may initialise it immediately and begin warehouse-based refresh work. Do not execute a production-like definition in a trial account simply to inspect its SQL. Review privileges, warehouse name, region and expected runtime first.

Project reference: P087: Declarative pipelines using Dynamic Tables. *Not independently executed; rights review pending.*

8.5.3 Eligibility, source dependencies, and support checks

A Dynamic Table supports a broad subset of query expressions, but *incremental refresh* has stricter requirements than merely running an ordinary SELECT. Window functions, joins, aggregation, VARIANT operations, policies and input types need to be checked against the current supported-query matrix. For the reader's model, use this sequence: start with a minimal query, confirm explicit incremental refresh on a disposable object, inspect refresh history, and then incrementally introduce additional operators while preserving a known-good comparison result.

Snowflake documentation in October 2026 also describes ADAPTIVE refresh and custom incrementalisation capabilities. These are not interchangeable with INCREMENTAL: their reinitialisation conditions, support status and operational semantics require separate evaluation. This chapter deliberately uses INCREMENTAL to make the lab's assumption explicit. An AUTO choice may resolve to FULL at creation, so it is not a reliable shorthand for an incremental performance promise.

Gate	Evidence required before promotion
Query support	Confirmed syntax and eligible operators for chosen refresh mode
Privileges	Owner role can read source and use approved refresh compute
Initialisation	Refresh status shows completion; consumers know when data is ready
Change volume	Sampled distribution of inserted, revised and deleted business keys
Correctness	Zero bidirectional set differences versus approved full-state reference
Cost	Measured warehouse/Cloud Services usage for bounded workload

Table 8.2. Declarative maintenance still needs an acceptance gate.

Subtle boundary. Dynamic Tables can support Snowflake-managed source tables and supported views; not every external, directory or specialised table structure is an eligible input. A pipeline involving side effects, external API calls or procedural branching may need a Task, a connector, or a hybrid architecture. Choosing one technology for every stage is not a goal.

> **Correctness before optimisation**
> A Dynamic Table refresh that costs fewer credits but chooses the wrong revision is still defective. Hold the input fixture, business key, tie-break order and assertions constant while comparing execution mechanisms.

Check. What does a successful CREATE DYNAMIC TABLE prove? What additional evidence is required before claiming that incremental refresh stays within a 20-minute business freshness objective?

Project reference: P088: Pipelines for agent workflows (advanced reference). *Not independently executed; rights review pending.*

8.6 Refresh modes, target lag, and dependencies

8.6.1 Target lag is a goal, not a cron expression

For a Dynamic Table, `TARGET_LAG='20 minutes'` asks Snowflake to maintain a certain freshness relationship between the materialised output and its source. It does not mean "run exactly every 20 minutes," nor is the target a guaranteed bound. If refresh work takes longer than the declared interval, actual staleness may exceed it. Teams should monitor the achieved freshness distribution and treat target lag as an optimisation input, not as a contractual latency measurement.

8.6.2 Incremental, full, auto, and adaptive choices

`INCREMENTAL` processes source changes according to eligible operators. `FULL` recomputes the result; it can be appropriate when most rows change each cycle or unsupported operators are required. `AUTO` selects a mode at creation and does not continually reconsider the choice. The current documentation also describes `ADAPTIVE`, which can favour incremental work while permitting a reinitialisation when bulk changes make it more economical; confirm availability and preview/GA status for the account at print time. No mode means *zero cost*; change detection and scheduling can have metering implications even when output rows do not change.

8.6.3 Pipeline graphs and the DOWNSTREAM setting

The target lag of an intermediate Dynamic Table can be `DOWNSTREAM`, allowing a downstream consumer with an explicit lag to drive its refresh timing. A terminal table with `DOWNSTREAM` and no dynamic consumer may never refresh automatically. Conversely, a downstream table cannot demand a smaller explicit lag than an upstream dependency allows. When a multi-stage graph is configured correctly, Snowflake coordinates refresh ordering; the designer must still validate end-to-end lag measured from the ultimate source.

Listing 7: Inspection after creation; outputs require interpretation.

```
SHOW DYNAMIC TABLES LIKE 'CH08_DT_LATEST';
-- Check refresh_mode, scheduling_state and lag.
-- For a separate downstream model:
-- TARGET_LAG = DOWNSTREAM on intermediate only.
```

Selection exercise
The dashboard is read once per hour; operators need validated measurements no older than 20 minutes. Give the consumer table an explicit 20-minute goal. Before shortening it to one minute, measure refresh duration and change-detection costs; a faster target has no value if the upstream provider publishes hourly.

Project reference: P089: Introductory Dynamic Tables reference. *Not independently executed; rights review pending.*

8.7 Dynamic Tables versus Streams/Tasks versus views

8.7.1 Imperative boundaries and declarative results

The key architectural distinction is *who describes the update procedure.* A Stream plus Task explicitly states when to run an operation and how to modify the target. A Dynamic Table states what the target relation should contain and delegates refresh planning to Snowflake. A standard View stores SQL text and computes its result when queried, usually without persisting a separately refreshed result. Each can be appropriate for a different consumption profile.

8.7.2 Compare constraints, not feature labels

Dimension	Stream + Task	Dynamic Table
Business logic	Explicit DML/procedure	SQL query definition
State writes	Target is mutable	Managed output read-only
Scheduling	Task schedule/stream gate	Target-lag refresh orchestration
Retry boundary	Task and transaction design	Managed refresh and history
Side effects	Possible via supported procedural tools	Not in declarative query
Recovery	Stream offset + target + archive	Source replay + refresh/reinitialise
Cost evidence	Task run, warehouse, checks	Refresh warehouse, service work

Table 8.3. Decision criteria for maintained transformations.

A Stream plus Task is often natural when a durable `MERGE` target must be updated conditionally, when several statements form an approved transaction, or when a downstream service requires an explicit action. A Dynamic Table is often more maintainable for a pure SQL relation that should track evolving base data. A View can be simplest when the dataset is small, reading is infrequent, or the cost of query-time computation is acceptable.

An important refinement: different mechanisms may be combined. For example, a Dynamic Table can prepare a validated intermediate relation while a triggered Task performs imperative downstream publishing, subject to the documented stream-on-dynamic-table restrictions and the actual refresh mode. A hybrid system should have a crisp ownership boundary rather than two unsynchronised jobs modifying the same target.

> **Decision principle**
> Choose the lowest-maintenance approach that expresses the required semantics, meets a measured freshness objective, and permits transparent failure recovery. Do not migrate a working procedural pipeline merely because a newer declarative feature exists.

Check. Which pattern would you use for (a) a frequently read hourly energy summary, (b) a workflow that conditionally calls an external incident service, and (c) a one-off analyst query?

8.7.3 Maintenance, operational cost, and the hidden work

A useful comparison accounts for more than the number of lines in a SQL definition. Streams/Tasks can require stream-lag monitoring, Task retry policies, warehouse sizing, execution-order controls and manual recovery procedures. Dynamic Tables shift some of that orchestration into a managed service, but still require source-quality management, refresh troubleshooting, ownership privileges, business freshness alerts and explicit handling of unsupported operators. Views avoid refresh compute but move work to every eligible query. Total ownership cost depends on workload, staffing, and failure frequency as well as Snowflake charges.

For a controlled experiment, hold constant: source rows, row sizes, key cardinality, arrival rate, corrections, deletes, source retention and customer query frequency. Measure *equivalent* business results before costs. Then examine task compute credits; dynamic-table refresh compute and Cloud Services; bytes scanned; staleness distribution; retries; and operational response time. Use the account's official metering views with reporting lag disclosed. A resource monitor that suspends a virtual warehouse is not a universal cap on every serverless service charge.

Question	Measurement	Required disclosure
Correct result?	Bidirectional `MINUS`; sum	Null and delete semantics
Fresh enough?	p95/p99 event to silver	Clock boundaries; missing keys
Efficient?	Credits / 1M accepted facts	Full cost categories and volume
Recoverable?	Simulated outage and backfill	Time, data-loss boundary, run IDs
Maintainable?	Incidents + engineer effort	Workload and observation window

Table 8.4. A reproducible comparison checklist.

False economy example
A 60-second target on an unchanged hourly table can create frequent change-detection overhead with no additional business information. A low-lag feature setting should follow a user requirement and actual refresh evidence, not serve as a speed badge in documentation.

Security and governance. Ownership and execution privileges differ by resource. Tasks execute according to their task ownership and security context; Dynamic Tables refresh using their configuration and owner access. Privilege drift, revoked source grants or a changed masking policy can affect downstream refresh behaviour. Record ownership, permitted data visibility and alert routing at deployment rather than relying only on successful initial CREATE statements.

Project reference: P090: dbt and Snowflake pipeline deployment reference. *Not independently executed; rights review pending.*

8.8 Recovery, backfills, and pipeline observability

8.8.1 Designing backfills that preserve history

When a pipeline breaks, the first instinct is often to rerun yesterday's job. That may recover an append-only file load, but an incremental change consumer requires more care: what source interval is missing, which stream offset was committed, and which corrections arrived after the failure? A backfill must reconstruct an *approved state*, not merely append old events for a second time. Historical repair should also preserve evidence of the original run and the reason for reprocessing.

For the Finland energy case, use a repair ledger recording provider, UTC interval bounds, source snapshot hash or event offsets, intended target keys, invocation ID and post-repair assertions. Stop affected publishing before repair if readers could see inconsistent mixed versions. With Streams and Tasks, a prolonged outage may require rebuilding target state from the trusted bronze archive and then establishing a fresh, valid stream offset. With Dynamic Tables, options include resuming failed refresh, manual refresh, or approved recreation/reinitialisation depending on source and schema changes. A full refresh is not a guarantee of recovery if the source archive is incomplete.

Seven-step incident playbook

1. Confirm the incident and freeze further destructive changes.
2. Capture SQL/query IDs, Task history, stream state and dynamic refresh history.
3. Define expected source cutoff and acceptance invariants.
4. Select authoritative event archive or snapshot.
5. Rebuild in isolation and compare against a deterministic oracle.
6. Publish only after acceptance checks pass and monitoring is restored.
7. Preserve the incident log, access approvals and cost record.

Backfill example. A publisher corrects the 10:00 wind measurement from revision 1 to revision 2 and a 10:05 downstream run fails. If the source event is still available, successful retry must publish 14 MWh exactly once. If the stream is stale, it is not safe simply to create a new stream and assume that the unseen historical correction will reappear. Reconstruct the silver state from retained accepted history first.

Recovery time objective versus data-loss objective

An RTO of one hour says how rapidly useful service should return. An RPO of zero accepted observations says what state cannot be lost. A design without adequate source history may meet one objective while failing the other. State both in the architecture decision record.

Check. What must be proven before a repaired silver table replaces the current production table? How does the answer change if source corrections can arrive a day late?

8.8.2 Freshness regression and silent failure detection

A pipeline may be "green" while still publishing old or incomplete data. A Task may finish successfully but process zero approved rows because its source was empty, a filter was wrong, or every input failed validation. A Dynamic Table may be running but miss its target lag. Measure multiple independent signals: *source high-watermark*, *last accepted bronze event*, *last published silver interval*, *age of oldest missing correction*, and *quarantine rate*. A sudden drop in valid output is an incident even when system errors remain at zero.

The key alert for a provider should distinguish no fresh source data from delayed transformation. Compute $L = t_{\text{observation}} - t_{\text{latest approved source event}}$ as a business signal with documented clocks. Track an additional event-to-visible measure for individual records. If a provider legitimately publishes hourly, a 20-minute target for the refresh mechanism does not make the public dataset itself up-to-the-minute. Avoid reporting the tool's scheduler setting as achieved freshness.

8.8.3 Lineage, Query History, and audit evidence

Record the chain: source file or event ID → bronze key/revision → stream consumption or dynamic refresh → silver state → dashboard query. Snowflake Query History, Task History and Dynamic Table refresh history can help explain a failure, subject to privileges and reporting latency. Capture the relevant IDs and timestamps in the incident log, and keep a narrow reproducible diagnostic SQL query.

Listing 8: Illustrative history inspections; permission requirements apply.

```
SELECT * FROM TABLE(
  INFORMATION_SCHEMA.TASK_HISTORY(
    TASK_NAME => 'CH08_ENERGY_MERGE_TASK',
    RESULT_LIMIT => 20));
SHOW DYNAMIC TABLES LIKE 'CH08_DT_LATEST';
SHOW STREAMS LIKE 'CH08_BRONZE_STREAM';
```

Refresh and Task history fields can change; validate the approved account version before copying these queries into operational dashboards. Do not confuse delayed account-usage reporting with absence of cost. Never disclose customer data or credentials in screenshots or monitoring exports intended for a public repository.

Three complementary alerts
Freshness: one source's last approved interval is older than its service objective. **Correctness:** business-key duplicates, invalid revisions or aggregation mismatch appear. **Liveness:** a Task or refresh fails repeatedly, or the stream approaches staleness. Suppressing one must not silence the others.

Project reference: P091: Snowflake-Labs engineering toolbox. *Not independently executed; rights review pending.*

Project reference: P055: Alert Hub reference. *Not independently executed; rights review pending.*

8.9 Incremental transformation design exercise

8.9.1 Choose a pattern for two distinct workloads

Workload A, energy dashboard: immutable revision events arrive irregularly, visible rows represent the latest approved state, consumers read results every ten minutes, and SQL can describe the transformation. Prefer a Dynamic Table if its operators are eligible, refresh cost is acceptable, and measured output freshness meets the business requirement. Choose Streams/Tasks if an explicit, tightly controlled mutable-state transaction or a side-effecting approval step is necessary.

Workload B, incident response: a corrected high-voltage alarm must be enriched, deduplicated, written to a ledger and then passed through an external action with an acknowledgment. A declarative table alone is not an adequate workflow engine. Keep a transactional state boundary and an independently idempotent outbound action; a Task or external orchestrator is a better fit for procedural steps.

8.9.2 Acceptance contract

For either workload, state a business key, tie-break order, deletion policy, invalid-event disposition, latest acceptable data age, maximum recovery time, archival retention, exact downstream exposure point and cost measurement method. An assertion such as "all records processed" is too weak: define exactly which records are accepted and how corrections replace earlier values. Require the two result implementations to agree under replay, late arrival, deletion and intentional fault injection.

8.9.3 Correctness, latency, and cost compared fairly

Use the same synthetic event history to prove logical equivalence offline. Then, only after authorisation, run both Snowflake implementations over a controlled cloud fixture, collecting query IDs, refresh histories and actual credits. Distinguish measured percentiles from arbitrary *target lag* settings; report all source and silver timestamps in UTC and clearly explain the measurement clock.

> **Chapter argument**
> The real choice is not "Streams or Dynamic Tables?" It is "Which responsibility should our team own, and which should Snowflake manage, without surrendering correctness, evidence or recovery?" Choose the implementation whose risks you can test and explain.

Further reading, verified technical documentation (22 October 2026): Streams; stream staleness; triggered tasks; MERGE; dynamic refresh modes; target lag; and eligible constructs. Availability, costs, previews and behaviour require another check before release.

Project reference: P092: Community Dynamic Table comparison. *Not independently executed; rights review pending.*

Project reference: P093: dbt, Streams and Tasks exploration. *Not independently executed; rights review pending.*

8.L Guided lab: two ways to maintain energy state

8.L.1 Prerequisites, inputs, permissions, and cost controls

The companion archive provides an event fixture, Python simulator and tests, procedural and declarative SQL templates, plus a comparison memo. Python 3.10+ and the standard library suffice for the *offline* portion. No cloud account, credentials, network or billed resources are required. The Snowflake scripts are integration templates and have *not* been executed in Snowflake. Before any optional cloud test, obtain approval for a disposable schema, a narrowly scoped role, a known warehouse and a fixed teardown plan.

Our fixture delivers 12 synthetic rows: six initial technologies, a wind correction, its repeated delivery, a temporary TEST measurement and subsequent tombstone, an old hydro revision, and one invalid negative solar measurement. The unique accepted event archive has ten events; the negative reading is quarantined. One event is a duplicate delivery. All observations use a UTC source interval, and the provider name is illustrative rather than a download from any real service.

8.L.2 Run the model and inspect the results

Listing 9: Reproducible offline commands from the companion lab directory.

```
python incremental_model.py
python -m unittest -v test_incremental_model.py
```

The script independently computes a full latest-state snapshot and a transactional-batch-style state reduction. The latter models one aborted attempt followed by a retry that must not lose data. Both outputs are compared after all accepted events, including a DELETE tombstone, have been evaluated. Files are written to `output/` as a JSON comparison ledger and small CSV exports.

Actually observed offline result	Value
Delivered rows / invalid quarantined	12 / 1
Unique accepted logical events / duplicate deliveries	10 / 1
Latest keys including one tombstone	7
Visible latest-revision energy facts	6
Approved energy total (MWh)	66.000
Simulated aborted batch subsequently retried	1
Procedural-model equals full snapshot	True

Table 8.5. Deterministic offline result, not a live Snowflake measurement.

Optional cloud path. Run the Table/Stream bootstrap and complete Task DDL in a permitted schema. The Task is intentionally left suspended. If it is explicitly enabled, its MERGE must commit before comparing the silver output against a completed Dynamic Table initialisation. A successful CREATE statement alone does not prove either data path completed.

8.L.3 Assertions, troubleshooting, rollback, and cleanup

Eleven offline tests exercise input accounting; duplicate event IDs; rejection of negative energy; corrected wind precedence; resistance to older replay; tombstone-based delete; energy sum; equality of full and incremental reductions; simulated failed-batch retry; replay idempotence; and the event-validation contract. A passing offline suite proves those *model* invariants for the supplied fixture, not live transaction or refresh guarantees.

If the sum is not 66.000 MWh, inspect the revision key and tie-break. If TEST reappears, confirm that its highest-version tombstone was preserved. If the two models diverge, reduce the event sequence to the first differing key and compare its full revision history. If a real Task fails, preserve the query ID, statement error, stream status and committed target contents before retry. If a Dynamic Table appears empty, first check that its initial refresh has completed and the owner role can read its sources.

Cleanup. Delete the local `output/` folder. SQL teardown statements are commented out, as are Task activation statements. Suspending active cloud tasks and dropping objects requires explicit approval and a record of outputs, credentials and consumption. Never call DROP on a shared schema based on a textbook instruction.

8.L.4 Exercises, architectural decision, and scoring

Foundation: Add a duplicate corrected-wind event with a new delivery timestamp but the same event ID. Show which counters change and why the approved sum does not. **Practitioner:** Add a higher-revision late hydro correction; prove the new sum changes once, and that a replay of the original revision cannot undo it. **Architecture task:** Recommend either Streams/Tasks, Dynamic Tables or a View for an hourly Finnish market dashboard and a side-effecting fault alarm. Specify the tested correctness invariant, freshness metric, cost guardrail and recovery action.

Assessment rubric (10 points)
Versioned business key and tombstone semantics (2); deterministic reconciliation (2); transactional or declarative refresh explanation (2); failure/replay evidence (2); cost, least privilege, monitoring and cleanup plan (2). A design with no rollback mechanism cannot receive full marks.

Chapter review. (1) What advances a stream offset? (2) Why can the stream-data predicate yield a false positive? (3) Why is TARGET_LAG not a cron schedule? (4) Why must MERGE deduplicate before matching? (5) Which tasks still require human design when a Dynamic Table maintains SQL output?

Looking ahead. Chapter 9 adds Snowpark Python for transformations that benefit from a programmatic DataFrame interface; the need for stable keys, validation, lineage and replay discipline remains unchanged.

Print-release verification gate
Before print, verify feature status, access, credits and external project rights. Test both SQL paths in an approved Snowflake account and retain query IDs, costs and teardown proof. No live-cloud execution is claimed.